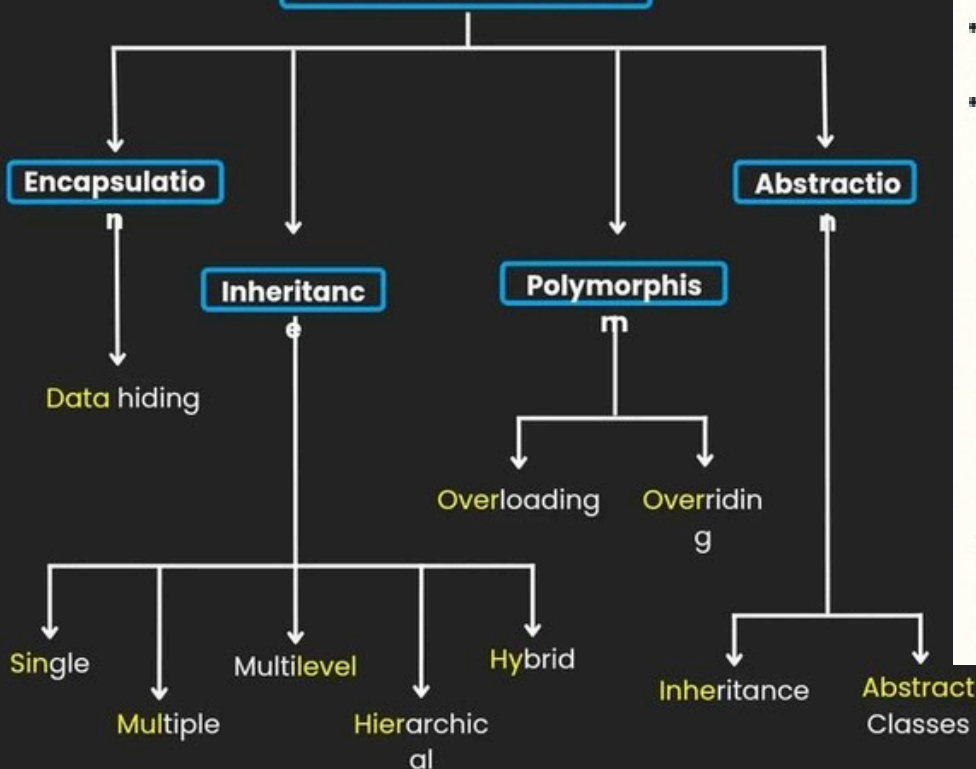




FAZAIA BILQUIS COLLEGE NUR KHAN BASE, RWP

OOP Introduction 🔥

Pillars of OOP



ENCAPSULATION

```
class Car
private void
start() {
stop() {
}
```



- HIDING INTERNAL DETAILS
- RESTRICTING ACCESS

ABSTRACTION



- SHOWING ESSENTIAL FEATURES
- HIDING COMPLEXITY

ASSIGNMENT # 04

NAME:

MARIA ATTA

ASSIGNMENT #:

OOPS _06

ROLL NO:

BSCS-13-F24-01

TEACHER:

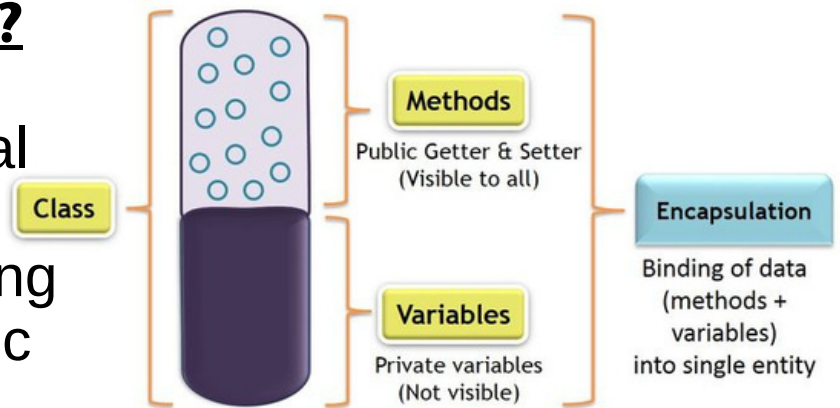
Ma'm SANA

LAB: 06 IMPLEMENTING ENCAPSULATION AND ABSTRACTION

• ENCAPSULATION – BANK ACCOUNT MANAGEMENT SYSTEM

What is Encapsulation?

Encapsulation means hiding a class's internal details and only allowing access through specific methods. It helps keep data safe and secure from outside changes.



Real-Life Example:

Think of a bank account. You can't directly change your balance or name. You can only deposit or withdraw through secure methods.

What Students Did:

- Created a BankAccount class.
- Kept details like name, account number, and balance private.
- Used getter/setter methods to access and update data.
- Added checks like no negative deposits and no overdrafts.

• ABSTRACTION – ONLINE PAYMENT SYSTEM

What is Abstraction?

Abstraction means showing only essential details and hiding the background work. It helps build clean and reusable code.



Real-Life Example:

When you pay online, you don't see how the system works inside. You just enter the card/UPI details and confirm payment.

What Students Did:

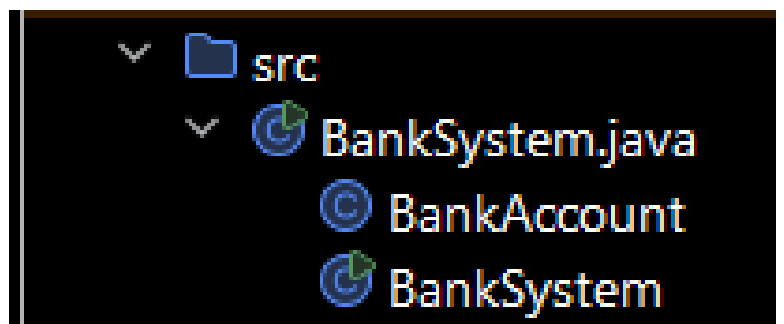
- Created an **abstract class** **Payment** with a method to set the amount and show the summary
- Added an **abstract method** **processPayment()** – different for each payment type
- Made two classes:
 - **CreditCardPayment** → Needs card number & CVV
 - **UPIPayment** → Needs UPI ID
- Used **polymorphism**: called different payment types using a common **Payment** reference.

SCENARIO 1: ENCAPSULATION – BANK ACCOUNT MANAGEMENT

Goal: Protect data inside a class and allow controlled access using methods.

Concepts Used:

- Private variables
- Public getter/setter methods
- Input validation



Task:

- Create a **BankAccount** class with:
- Private **fields**: **customerName**, **accountNumber**, and **balance**.

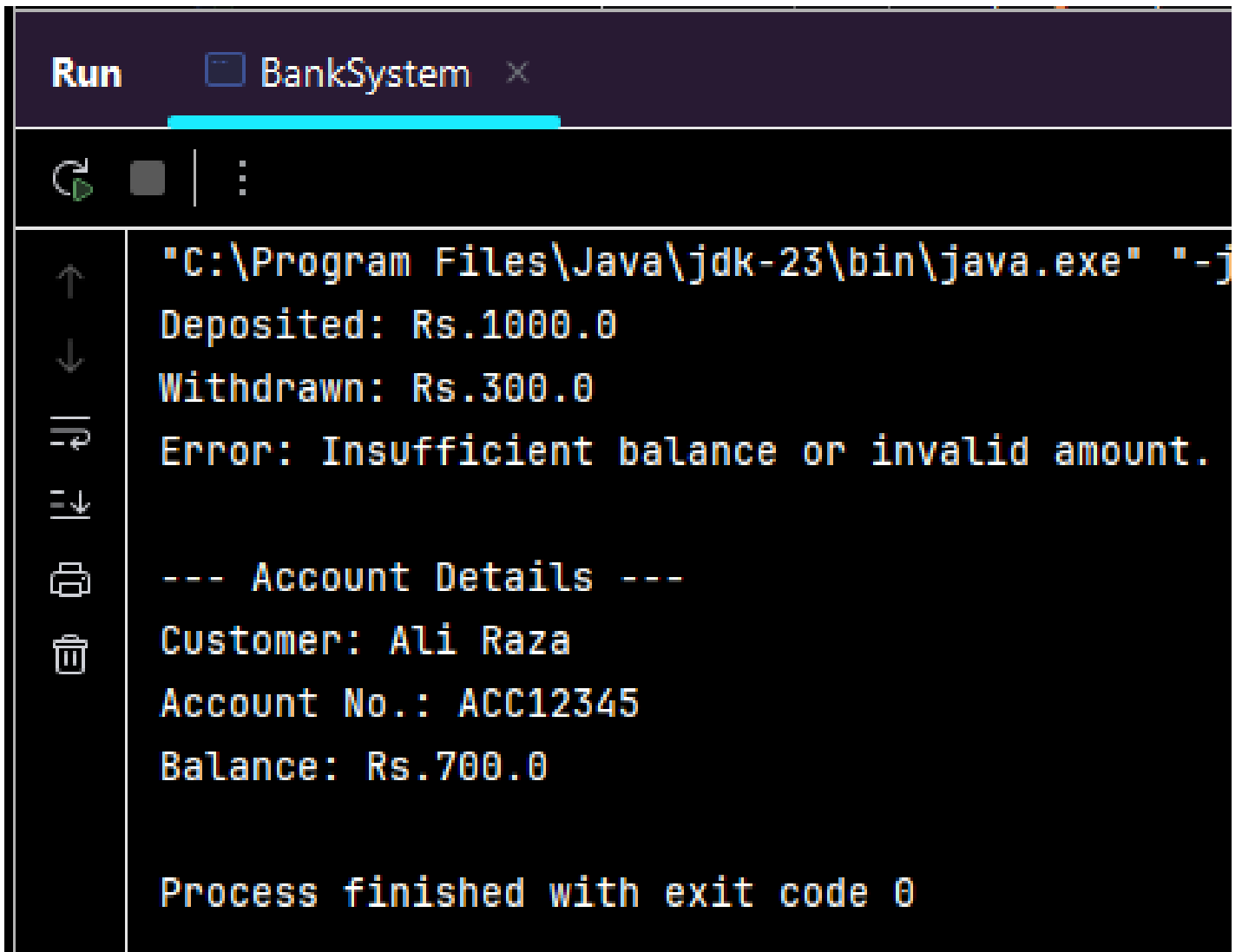
- **Methods to:**
 - **Deposit money** (reject negative amounts)
 - **Withdraw money** (only if balance is enough)
 - **View account details** (via a display method)
- **PROGRAM:**

```

BankSystem.java ×
1  class BankAccount { 2 usages
2      private String customerName; 2 usages
3      private String accountNumber; 2 usages
4      private double balance; 4 usages
5      // Set customer name
6  >  public void setCustomerName(String name) { customerName = name; }
9      // Set account number
10 >  public void setAccountNumber(String accNo) { accountNumber = accNo; }
13     // Deposit money (only positive amounts)
14     public void deposit(double amount) { 1 usage
15         if (amount > 0) {
16             balance += amount;
17             System.out.println("Deposited: Rs." + amount);
18         } else {
19             System.out.println("Error: Deposit amount must be positive.");
20         }
21     }
22     // Withdraw money (only if enough balance)
23     public void withdraw(double amount) { 2 usages
24         if (amount > 0 && amount <= balance) {
25             balance -= amount;
26             System.out.println("Withdrawn: Rs." + amount);
27         } else {
28             System.out.println("Error: Insufficient balance or invalid amount.");
29         }
30     }
31     // Display account info
32     public void displayAccountInfo() { 1 usage
33         System.out.println("\n--- Account Details ---");
34         System.out.println("Customer: " + customerName);
35         System.out.println("Account No.: " + accountNumber);
36         System.out.println("Balance: Rs." + balance);
37     }
38 }
39 > public class BankSystem {
40 >     public static void main(String[] args) {
41         BankAccount account = new BankAccount();
42         account.setCustomerName("Ali Raza");
43         account.setAccountNumber("ACC12345");
44         account.deposit(amount: 1000); // Valid deposit
45         account.withdraw(amount: 300); // Valid withdrawal
46         account.withdraw(amount: 800); // Fails (insufficient balance)
47         account.displayAccountInfo();
48     }
49 }

```

- OUTPUT:



```
Run BankSystem x
"C:\Program Files\Java\jdk-23\bin\java.exe" "-j
Deposited: Rs.1000.0
Withdrawn: Rs.300.0
Error: Insufficient balance or invalid amount.

--- Account Details ---
Customer: Ali Raza
Account No.: ACC12345
Balance: Rs.700.0

Process finished with exit code 0
```

- CODE EXPLANATION:

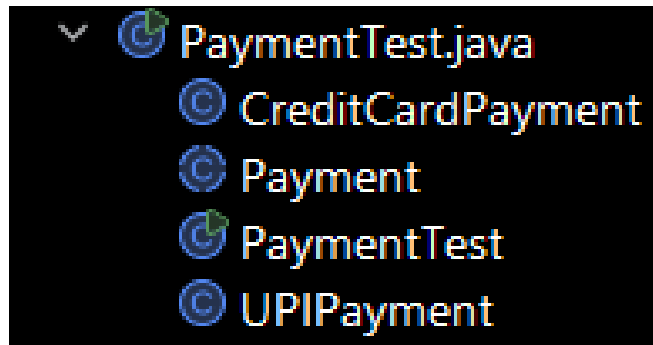
- **private:** Hides the variables so they can't be accessed directly from outside.
- **BankAccount(...):** Constructor – sets initial values when a new object is created.
- **deposit(...):** Adds money only if amount is greater than 0.
- **withdraw(...):** Subtracts money only if there's enough balance.
- **displayDetails():** Prints all account information in a readable format.

SCENARIO 2: ABSTRACTION – ONLINE PAYMENT SYSTEM

Goal: Use an abstract class to define a common structure for various payment methods.

Concepts Used:

- Abstract class and method
- Inheritance
- Polymorphism



Task:

Create an abstract class **Payment** with:

- A method to set amount
- A method to show payment summary
- An abstract method **processPayment()** (implemented in subclasses)

Then, create **two subclasses**:

- **CreditCardPayment** – Takes card number and CVV
- **UPIPayment** – Takes UPI ID

• PROGRAM:

```
PaymentTest.java ×
1 // Abstract base class
2 abstract class Payment { 2 usages
3     protected double amount; 2 usages
4     public void setAmount(double amt) { 2 usages
5         amount = amt;
6     }
7     public void showSummary() { 2 usages
8         System.out.println("Amount to pay: Rs." + amount);
9     }
10    // Abstract method (must be implemented by subclasses)
11    public abstract void processPayment(); 2 usages 2 implementations
12 }
13 // Credit Card Payment
14 class CreditCardPayment extends Payment { 2 usages
15     private String cardNumber; 2 usages
16     private String cvv; 2 usages
17     public void setCardDetails(String card, String cvvCode) { 1 usage
18         cardNumber = card;
```

```

19         cvv = cvvCode;
20     }
21     @Override 2 usages
22     public void processPayment() {
23         System.out.println("\nProcessing Credit Card Payment:");
24         System.out.println("Card: " + cardNumber);
25         System.out.println("CVV: " + cvv);
26         showSummary();
27     }
28 }
29 // UPI Payment (additional subclass example)
30 class UPIPayment extends Payment { 2 usages
31     private String upiId; 2 usages
32     public void setUpiId(String id) { 1 usage
33         upiId = id;
34     }
35     @Override 2 usages
36     public void processPayment() {
37         System.out.println("\nProcessing UPI Payment:");
38         System.out.println("UPI ID: " + upiId);
39         showSummary();
40     }
41 }
42 public class PaymentTest {
43     public static void main(String[] args) {
44         // Credit Card Payment
45         CreditCardPayment creditCard = new CreditCardPayment();
46         creditCard.setAmount(1500);
47         creditCard.setCardDetails(card: "1234-5678-9876-5432", cvvCode: "123");
48         creditCard.processPayment();
49         // UPI Payment
50         UPIPayment upi = new UPIPayment();
51         upi.setAmount(500);
52         upi.setUpiId("ali@upi");
53         upi.processPayment();
54     }
55 }

```

• OUTPUT:

```

↑ "C:\Program Files\Java\jdk-23\bin\java.exe"
↓
Processing Credit Card Payment:
|| Card: 1234-5678-9876-5432
-~ CVV: 123
=↓ Amount to pay: Rs.1500.0
☐
☑ Processing UPI Payment:
UPI ID: ali@upi
Amount to pay: Rs.500.0

```


- **CODE EXPLANATION:**
- **abstract class Payment:** Can't be used directly, but gives a base structure for all payments.
- **setAmount(...):** Sets how much is to be paid.
- **showSummary():** Displays the payment amount.
- **processPayment():** Abstract method – subclasses must define it their own way.
- **CreditCardPayment:** Has extra details like card number and CVV.
- **UPIPayment:** Has UPI ID and its own payment logic.

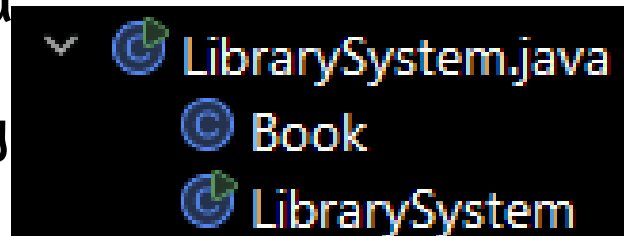
HOME TASKS

HOME TASK 1: LIBRARY BOOK MANAGEMENT SYSTEM (ENCAPSULATION)

Scenario: You are developing a digital library. Each book has a **title**, **ISBN**, and **availability status**. Users shouldn't directly access or change these values. Instead, safe methods should handle all updates.

Student Task:

1. Create a **Book** class with private fields: **title**, **isbn**, **isAvailable**.
2. Use public setter methods with validation:
 - Title must not be empty
 - ISBN must be at least 10 characters
3. Use getter methods to read values
4. Display full book information using a method
5. In **main()**, create an object, set values, and show details.



```

LibrarySystem.java
Book
LibrarySystem
  
```


• PROGRAM:

```
LibrarySystem.java x
1 class Book { 2 usages
2     private String title; 3 usages
3     private String isbn; 3 usages
4     private boolean isAvailable; 3 usages
5     // Setters with validation
6 @ public void setTitle(String title) { 1 usage
7     if (!title.isEmpty()) {
8         this.title = title;
9     } else {
10        System.out.println("Error: Title cannot be empty!");
11    }
12 }
13 @ public void setIsbn(String isbn) { 1 usage
14     if (isbn.length() >= 10) {
15         this.isbn = isbn;
16     } else {
17         System.out.println("Error: ISBN must be 10+ characters!");
18     }
19 }
20 > public void setIsAvailable(boolean status) { isAvailable = status; }
23 // Getters
24 public String getTitle() { return title; } no usages
25 public String getIsbn() { return isbn; } no usages
26 public boolean getAvailability() { return isAvailable; } no usages
27 // Display book info
28 public void displayBookInfo() { 1 usage
29     System.out.println("\n--- Book Details ---");
30     System.out.println("Title: " + title);
31     System.out.println("ISBN: " + isbn);
32     System.out.println("Status: " + (isAvailable ? "Available" : "Issued"));
33 }
34 }
35 ▶ public class LibrarySystem {
36 ▶     public static void main(String[] args) {
37         Book book = new Book();
38         book.setTitle("Java Programming");
39         book.setIsbn("1234567890");
40         book.setIsAvailable(true);
41         book.displayBookInfo();
42     }
43 }
```

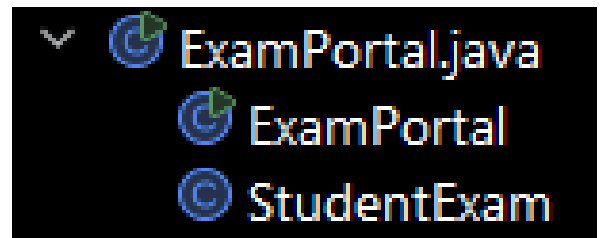
• OUTPUT:

```
↑ "C:\Program Files\Java\jdk-23\bin\java.exe"
↓
⇅ --- Book Details ---
⇅ Title: Java Programming
⇅ ISBN: 1234567890
⇅ Status: Available
```

- CODE EXPLANATION:
- abstract class Payment: The Book class hides data with private fields.
- Setters check if input is valid before setting values.
- displayInfo() prints the book details.
- Direct access is not allowed—only methods handle updates.

HOME TASK 2: ONLINE EXAM PORTAL (ENCAPSULATION)

Scenario: An online exam system stores **student name**, **exam ID**, and **score**. Score must be between 0–100. Based on the score, the system shows whether the student **Passed** or **Failed**.



Student Task:

1. Create **StudentExam** class with private fields
2. Use setters with validation
3. Use getters to access data
4. Add a method to display student info and result
5. Use **main()** to test your code

- PROGRAM:

```
ExamPortal.java ×
1  class StudentExam { 2 usages
2      private String studentName; 2 usages
3      private String examId; 2 usages
4      private int score; 3 usages
5      // Setters with validation
6  @ public void setStudentName(String name) { 1 usage
7      |     if (!name.isEmpty()) {
8      |         studentName = name;
9      |     } else {
10     |         System.out.println("Error: Name cannot be empty!");
11     |     }
12     }
```

```

13 @      public void setExamId(String id) { 1 usage
14         if (!id.isEmpty()) {
15             examId = id;
16         } else {
17             System.out.println("Error: Exam ID cannot be empty!");
18         }
19     }
20     public void setScore(int score) { 1 usage
21         if (score >= 0 && score <= 100) {
22             this.score = score;
23         } else {
24             System.out.println("Error: Score must be 0-100!");
25         }
26     }
27     // Display result
28     public void displayResult() { 1 usage
29         System.out.println("\n--- Exam Result ---");
30         System.out.println("Student: " + studentName);
31         System.out.println("Exam ID: " + examId);
32         System.out.println("Score: " + score);
33         System.out.println("Result: " + (score >= 50 ? "PASS" : "FAIL"));
34     }
35 }
36 public class ExamPortal {
37     public static void main(String[] args) {
38         StudentExam exam = new StudentExam();
39         exam.setStudentName("Ali Ahmed");
40         exam.setExamId("EXAM001");
41         exam.setScore(85);
42         exam.displayResult();
43     }
44 }

```

• OUTPUT:

Run

ExamPortal

×

↺

■

📷

📄

⋮

↑

↓

↶

↷

🖨

🗑

"C:\Program Files\Java\jdk-23\bin\java.exe"

--- Exam Result ---

Student: Ali Ahmed

Exam ID: EXAM001

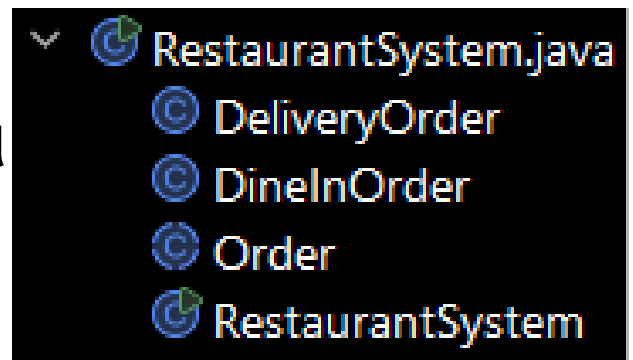
Score: 85

Result: PASS

- **CODE EXPLANATION:**
- Validations check that name and ID are not empty, and score is within range.
- The result is decided using a condition: $\text{score} \geq 50 \rightarrow \text{Pass}$.
- All fields are private to prevent misuse.

HOME TASK 3: RESTAURANT ORDERING SYSTEM (ABSTRACTION)

Scenario: A restaurant has **Dine-In** and **Delivery orders**. Both need customer name and item, but their bill calculation is different. Dine-In adds **service charge**, Delivery adds **delivery fee**.



Student Task:

1. Create an abstract **Order** class with **customerName**, **item**, and **setOrder()**
2. Add abstract method **calculateBill()**
3. Make subclasses:
 - **DineInOrder** → adds service charge
 - **DeliveryOrder** → adds delivery fee
4. Use both in **main()** with polymorphism

• **PROGRAM:**

```

RestaurantSystem.java ×
1  abstract class Order { 4 usages
2      protected String customerName; 1 usage
3      protected String item; 1 usage
4      public void setOrderDetails(String name, String item) { 2 usages
5          customerName = name;
6          this.item = item;
7      }
8      public abstract double calculateBill(); 2 usages 2 implementations
9  }

```

```

10 class DineInOrder extends Order { 1 usage
11     @Override 2 usages
12     public double calculateBill() {
13         double basePrice = 500; // Example price
14         return basePrice + (basePrice * 0.1); // +10% service charge
15     }
16 }
17 class DeliveryOrder extends Order { 1 usage
18     @Override 2 usages
19     public double calculateBill() {
20         double basePrice = 500;
21         return basePrice + 50; // +50 delivery fee
22     }
23 }
24 public class RestaurantSystem {
25     public static void main(String[] args) {
26         Order dineIn = new DineInOrder();
27         dineIn.setOrderDetails(name: "Ali", item: "Burger");
28         System.out.println("Dine-In Bill: Rs." + dineIn.calculateBill());
29         Order delivery = new DeliveryOrder();
30         delivery.setOrderDetails(name: "Hina", item: "Pizza");
31         System.out.println("Delivery Bill: Rs." + delivery.calculateBill());
32     }
33 }

```

• OUTPUT:

```

Run RestaurantSystem
"C:\Program Files\Java\jdk-23\bin\java.exe"
Dine-In Bill: Rs.550.0
Delivery Bill: Rs.550.0
Process finished with exit code 0

```

• CODE EXPLANATION:

- **Order** is abstract, can't be used directly.
- Each subclass defines its own bill method.
- Different charges are added based on order type.

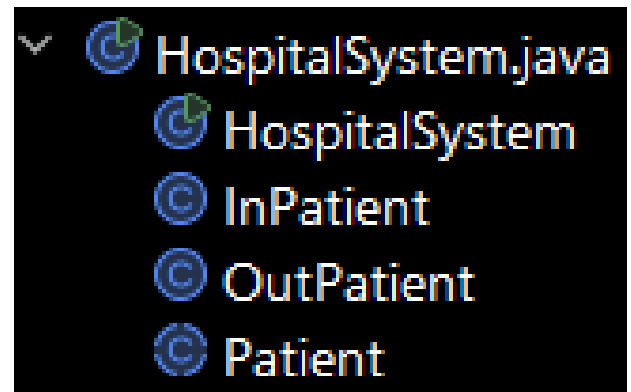
HOME TASK 4: HOSPITAL MANAGEMENT SYSTEM (ABSTRACTION)

Scenario: A hospital has two types of patients: **In-Patient** and **Out-Patient**. In-Patients pay for room + treatment, while Out-Patients pay for consultation only.

Student Task:

1. Create abstract class **Patient** with fields and a method to set details
2. Add abstract method **generateBill()**
3. Make two subclasses:
 - **InPatient** → room + treatment
 - **OutPatient** → consultation
4. Create objects in **main()** and use polymorphism

• **PROGRAM:**



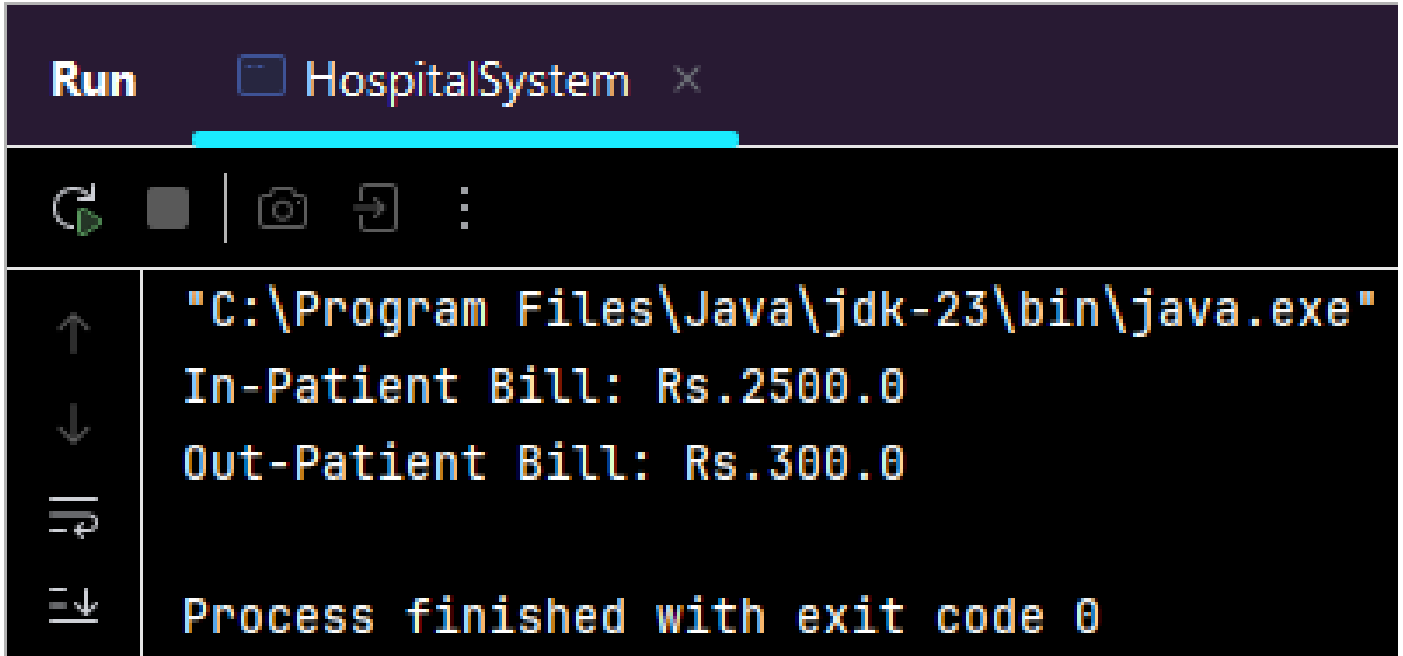
```
HospitalSystem.java x
1  abstract class Patient { 4 usages 2 inheritors
2      protected String name; 1 usage
3      protected String patientId; 1 usage
4      public void setDetails(String name, String id) { 2 usages
5          this.name = name;
6          patientId = id;
7      }
8      public abstract double generateBill(); 2 usages 2 implementations
9  }
10 class InPatient extends Patient { 1 usage
11     @Override 2 usages
12     public double generateBill() {
13         return 2000 + 500; // Room + treatment charges
14     }
15 }
16 class OutPatient extends Patient { 1 usage
17     @Override 2 usages
18     public double generateBill() {
19         return 300; // Consultation only
20     }
21 }
22 public class HospitalSystem {
23     public static void main(String[] args) {
24         Patient inpatient = new InPatient();
25         inpatient.setDetails(name: "Ali", id: "P001");
```

```

26     System.out.println("In-Patient Bill: Rs." + inpatient.generateBill());
27     Patient outpatient = new OutPatient();
28     outpatient.setDetails(name: "Hina", id: "P002");
29     System.out.println("Out-Patient Bill: Rs." + outpatient.generateBill());
30 }
31 }

```

• OUTPUT:



```

Run HospitalSystem x
"C:\Program Files\Java\jdk-23\bin\java.exe"
In-Patient Bill: Rs.2500.0
Out-Patient Bill: Rs.300.0
Process finished with exit code 0

```

• CODE EXPLANATION:

- **Patient** gives common structure.
- **InPatient** and **OutPatient** calculate bills differently.
- Abstract class is used for **code reuse** and **polymorphism**.